

M2. Les listes avancées



Dans ce module sur Python 3, nous aborderons les sujets suivants liés aux listes :

1. **Iterable et iterator** : vous apprendrez comment parcourir les éléments d'une liste de manière efficace à l'aide de boucles for ou de fonctions telles que `iter()` et `next()`.
2. **Sets** : vous découvrirez les ensembles non ordonnés de valeurs uniques, qui sont utiles pour effectuer des opérations ensemblistes et éliminer les doublons.
3. **Tuples** : vous comprendrez l'utilisation des tuples, similaires aux listes mais immuables, dans des cas spécifiques où vous ne souhaitez pas modifier les éléments.
4. **Dictionnaires** : vous explorerez les dictionnaires, des structures de données associatives permettant de stocker des paires clé-valeur.

Ce module vous permettra de maîtriser ces aspects avancés des listes en Python 3 pour améliorer votre programmation et vos compétences en développement Python.



 Les sets Les tuples Les dictionnaires Review

Notion d'itérable et d'iterator

Il est essentiel de comprendre les concepts d'**itérable** et d'**iterator** lorsqu'on manipule des données séquentielles telles que des collections ou des chaînes de caractères.

Ces deux notions sont étroitement liées, et il est courant pour les débutants en programmation de les confondre et de ne pas comprendre leurs utilisations respectives.

L'itérable

Un itérable est un **objet qui peut être parcouru séquentiellement**, ce qui signifie que vous pouvez accéder à ses éléments un par un.

Les listes, les tuples, les chaînes de caractères et les dictionnaires sont des exemples courants d'itérables. Vous pouvez utiliser un itérable dans une boucle for pour parcourir et traiter ses éléments.

À ce stade, **vous devez déjà tous connaître l'itérable le plus courant : la liste.**

```
maList = ["Je", "suis", "un", "itérable"]  
for value in maList:
```

```
print (value)
```

Code qui affiche les différents items d'une liste.

Comme attendu, ce script donne un résultat prévisible. Les informations suivantes seront affichées dans la console de débogage :

```
Je  
suis  
un  
iterable
```

Résultat de l'exécution du script.

L'iterator

Un **iterator** est un objet servant à **parcourir séquentiellement un élément itérable**.

Un tel iterator conserve en mémoire un pointeur interne, qui dirige vers l'élément suivant à renvoyer à chaque nouvelle itération.



Un **pointeur interne** se réfère à une variable spéciale qui contient l'adresse mémoire d'un autre élément.

Pour instancier un iterator, il faut faire appel à la méthode `iter()`, qu'on doit appliquer à une instance d'itérable.

Une fois l'iterator en main, la méthode `next()` sera utilisée pour accéder à l'élément suivant de l'itération.

Si l'iterator tente de parcourir au-delà de la fin de l'itérable et qu'aucun élément supplémentaire n'est disponible, il génère une exception **StopIteration**.

Il est donc de la responsabilité du développeur, lors de l'utilisation manuelle d'un iterator, de gérer la situation où l'on atteint la fin du parcours.

Considérez cet exemple initial où une chaîne de caractères est traitée comme une séquence :

```
maSequence = "Je suis une séquence"
monIterator = iter(maSequence)

index = 0

while index < len(maSequence):
    print(next(monIterator))
    index = index + 1
else:
    print("Fin de traitement")
```

Code d'un script qui traite une chaîne de caractères comme une séquence.

L'exécution du script affichera le résultat suivant :

```
J
e

s
u
i
s

u
n
e

s
é
q
u
e
n
c
e
Fin de traitement
```

Résultat du code précédent.

Notre second exemple exécutera le même traitement, mais en utilisant un iterator provenant d'une séquence de type liste :

```
maSequence = ["Ma", "sequence", "de", "fou"]
monIterator = iter(maSequence)

index = 0

while index < len(maSequence):
    print(next(monIterator))
    index = index + 1
```

```
else:  
    print("Fin de traitement")
```

Code d'un script utilisant un iterator provenant d'une liste.

Le résultat dans ce cas sera également sans surprise après exécution du script :

```
Ma  
sequence  
de  
fou  
Fin de traitement
```

Résultat de l'exécution du script.

En conclusion : les itérables et les itérateurs sont étroitement liés et sont souvent utilisés ensemble. Les itérables fournissent une interface permettant de parcourir séquentiellement les éléments tandis que les itérateurs maintiennent l'état de cette itération et retournent les éléments un par un. En utilisant les itérables et les itérateurs, vous pouvez parcourir efficacement les données séquentielles de manière contrôlée et optimisée.

CONTINUER

Les sets

Définition

Les listes ne sont pas les seules collections existantes en Python.

Les **sets** font partie de ces collections qu'il est possible de manipuler.

Contrairement à une liste, **un set est une collection non ordonnée d'éléments uniques.**

Les sets sont par ailleurs très utiles lorsque il est nécessaire de stocker des données sans se soucier ni de l'ordre ni de la duplication de celles-ci.

Création d'un set

Pour manipuler un set, la première chose à faire est de le créer. Deux méthodes peuvent être mises en œuvre.

1° Utilisation des accolades


```
monSet = {'pomme', 'orange', 'banane'}  
print(monSet)
```

Script qui crée et affiche un set.

Le résultat de ce script sera l'affichage des données telles que nous les avons encodées :

```
{'pomme', 'orange', 'banane'}
```

Le code suivant produira un résultat identique, suivant la définition du set, **les doublons n'étant pas intégrés ni intégrables à la séquence.**

```
monSet = {'pomme', 'orange', 'banane', 'banane'}  
print(monSet)
```

Autre script qui crée et affiche un set (sans doublons).

2° Utilisation de la fonction set()

Sur base d'une liste nous pouvons utiliser la fonction set() afin de **convertir la liste en set** et lui appliquer les règles régissant le fonctionnement des sets.

```
maList = ['pomme', 'orange', 'banane']
monSet = set(maList)
print(monSet)
```

Script qui convertit une liste en set.

Le résultat de cette opération sera l'affichage des données telles que nous les avons encodées :

```
{ 'pomme', 'orange', 'banane' }
```

Le code suivant produira un résultat identique. Suivant le fonctionnement des sets, **les doublons ne seront pas intégrés ni intégrables à la séquence** ; la liste passée en paramètre sera donc filtrée afin d'en exclure les doublons.

```
maList = ['pomme', 'orange', 'banane', 'banane']
monSet = set(maList)
print(monSet)
```

Autre exemple de script qui convertit une liste en set.

Manipulation d'un set

Lorsqu'on travaille avec une collection, il est essentiel de pouvoir pratiquer un ensemble d'opérations de bases telles que :

- Ajouter un élément dans la collection.
- Supprimer un élément de la collection.
- Vérifier l'existence d'un élément dans la collection.
- Effectuer des opérations entre deux collections (fusion, comparaison, etc.).

Bon nombre d'opérations fonctionnant avec les listes fonctionnent également avec les sets : **pop()** qui vise à sortir un élément de la collection et **clear()** qui supprime l'ensemble des éléments de la collection.

Ajout d'un élément à une collection

Pour ajouter un élément à un set, on utilise la méthode **add()**.

Cette méthode fonctionne sur base du modèle suivant : **setAManipuler.add()**.

L'avantage de cette méthode est qu'aucune erreur ne sera rencontrée si l'élément existe déjà dans le set. De plus, aucun doublon ne sera ajouté à la collection.

Ajoutons l'entrée 'kiwi' au set précédemment créé :

```
monSet = {'pomme', 'orange', 'banane'}  
#on ajoute "kiwi" au set monSet  
monSet.add("kiwi")  
print(monSet)  
#on essaye d'ajouter une 2ème x "kiwi" au set monSet  
monSet.add("kiwi")  
print(monSet)
```

Script qui ajoute un élément à un set au moyen de la méthode `add()`.

Voici le résultat affiché après l'exécution de ce script:

```
{'pomme', 'kiwi', 'banane', 'orange'}  
{'pomme', 'kiwi', 'banane', 'orange'}
```

Le résultat du script précédent.

L'entrée 'kiwi' étant présente dans le set après la première opération **`add()`**, la deuxième opération sera donc **sans effet**.

Suppression d'un élément à une collection

Il existe deux méthodes pour supprimer un élément d'un set :

- `remove()`
- `discard()`

Ces méthodes fonctionnent sur base du même modèle que pour la méthode `add()`:

`setAManipuler.remove('valeur')` ou `setAManipuler.discard('valeur')`.

La méthode **`remove()`** aura la fâcheuse tendance à **générer une exception** si l'entrée ciblée n'existe pas dans le set tandis que la méthode **`discard()`** **ne générera pas d'erreur** dans la même situation.

Supprimons la valeur 'kiwi' de notre set:

```
monSet = {'pomme', 'orange', 'banane', 'kiwi'}
#on supprime "kiwi" du set monSet
monSet.remove("kiwi")
print(monSet)
#on essaye de supprimer une 2ème x "kiwi" du set monSet
monSet.remove("kiwi")
print(monSet)
```

Script qui enlève des éléments d'un set avec la méthode `remove()`.

L'exécution de ce code nous renverra une erreur. En effet, lors du deuxième appel de `remove()`, étant donnée que la valeur a déjà été supprimée, le code lèvera une exception.

```
KeyError: 'kiwi'
```

Contrairement au code suivant qui ne renverra pas d'erreur du fait de l'utilisation de la méthode `discard()` :

```
monSet = {'pomme', 'orange', 'banane', 'kiwi'}
#on supprime "kiwi" du set monSet
monSet.remove("kiwi")
print(monSet)
#on essaye de supprimer une 2ème x "kiwi" du set monSet
monSet.discard("kiwi")
print(monSet)
```

Script qui enlève des éléments d'un set avec la méthode `discard()`.

Vérification de la présence d'un élément dans une collection

Pour vérifier la présence d'un élément dans une collection, on utilise l'opérateur [in](#).

Vérifions si 'kiwi' existe dans le set:

```
monSet = {'pomme', 'orange', 'banane', 'kiwi'}

if 'kiwi' in monSet:
    print("Kiwi existe dans le set")
else:
    print("Kiwi n'existe pas dans le set")
```

Script qui vérifie la présence d'un élément au moyen de l'opérateur `in`.

Lors de l'exécution du code, 'kiwi' faisant effectivement partie du set, notre affichage sera le suivant:

```
Kiwi existe dans le set
```

Résultat du script précédent.

Opérations entre collections

Nous avons vu comment ajouter un élément dans un set, supprimer un élément d'un set mais que faire dans le cas où nous souhaitons **traiter des collections de données** ?

1° L'opérateur | et la méthode union()

L'utilisation de l'opérateur | ou de la méthode union va permettre de **fusionner deux collections** en concentrant tous les éléments uniques dans une et une seule collection.

Reprenons l'exemple du set de fruits :

```
monSet = {'pomme', 'orange', 'banane', 'kiwi'}
monSetToAdd = {'mandarine', 'pêche'}

resultat1 = monSet | monSetToAdd
resultat2 = monSet.union(monSetToAdd)

print(resultat1)
print(resultat2)
```

Script qui fusionne des sets.

Vous remarquez que le résultat de l'exécution du code est bel et bien un nouveau set contenant les informations des deux sets utilisés dans l'opération d'union :

{'pomme', 'pêche', 'mandarine', 'banane', 'kiwi', 'orange'}

{'pomme', 'pêche', 'mandarine', 'banane', 'kiwi', 'orange'}

 Il est important de souligner que l'opération doit se faire sur base de collections de même type. Faute de types identiques, votre application lèvera une exception.

2° L'opérateur & et la méthode intersection()

L'utilisation de l'opérateur & ou de la méthode intersection() va permettre d'**extraire un set des données présentes dans les deux collections** au sein d'une et une seule collection.

Reprenons l'exemple de notre set de fruits :

```
monSet = {'pomme', 'orange', 'banane', 'kiwi'}
monSetToAdd = {'pomme', 'banane'}

resultat1 = monSet & monSetToAdd
resultat2 = monSet.intersection(monSetToAdd)

print(resultat1)
print(resultat2)
```


Script qui extrait des valeurs communes à deux sets.

Remarquez que le résultat de l'exécution du code est bel et bien un nouveau set contenant les informations communes aux deux sets utilisés dans l'opération d'intersection :

```
{ 'pomme', 'banane' }  
{ 'pomme', 'banane' }
```

Résultat du script précédent.

3° L'opérateur - et la méthode difference()

L'utilisation de l'opérateur - ou de la méthode difference() va permettre d'extraire un set de données exclusives à chacune des deux collections au sein d'une et une seule collection.

Reprenons l'exemple de notre set de fruits :

```
monSet = {'pomme', 'orange', 'banane', 'kiwi'}  
monSetToAdd = {'pomme', 'banane'}  
  
resultat1 = monSet - monSetToAdd  
resultat2 = monSet.difference(monSetToAdd)  
  
print(resultat1)  
print(resultat2)
```

Script qui extrait les valeurs uniques à un ensemble de sets.

Vous remarquez que le résultat de l'exécution du code est un nouveau set contenant les informations exclusives aux deux sets utilisés dans l'opération de différenciation :

```
{'orange ', 'kiwi '}  
{'orange ', 'kiwi '}
```

Résultat du script précédent.

CONTINUER

Les tuples

Définition

Les **tuples** sont sensiblement similaires aux listes.

Un tuple est une collection ordonnée et *immuable* d'éléments ; ce qui signifie que les tuples ne peuvent plus être modifiés une fois créés.

 Les objets dont la valeur peut changer sont dits muables (mutable en anglais) ; les objets dont la valeur est définitivement fixée à leur création sont dits immuables (immutable en anglais).

Création d'un tuple

Pour créer un tuple, **on fait usage de parenthèses** (et non de crochets comme pour les listes).

Exemple :

```
monTuple = ("pomme", "banane", "orange")  
print(monTuple)
```

Script qui crée et affiche un tuple.

Le résultat de l'exécution de notre code nous donnera le résultat suivant

```
('pomme', 'banane', 'orange')
```

Résultat du script précédent.

Il est possible de **créer un tuple à partir d'une liste** au moyen de la fonction **tuple()** :

```
monTuple = tuple(["pomme", "banane", "orange"])  
print("Ma liste convertie en tuple :", monTuple)
```

Le résultat de l'exécution de ce code sera le suivant :

```
Ma liste convertie en tuple : ('pomme', 'banane', 'orange')
```

Résultat de l'exécution du script précédent.

Le résultat de l'exécution de ce code sera le suivant :

Manipulation d'un tuple

Un tuple ne peut être modifié une fois créé mais il se manipule de la même manière qu'une liste : au moyen d'index.

L'index minimal d'un tuple est toujours **0**.

L'index maximal d'un tuple est quant à lui toujours égal à la longueur du tuple - 1.

 Pour rappel, la longueur d'une collection peut être obtenue au moyen de la fonction `len()`. Exemple : `len(monTuple)`

Dans l'exemple suivant, l'élément d'index 1 est affiché :

```
monTuple = ("pomme", "banane", "orange")
print(monTuple[1])
```

Script qui affiche l'élément d'index 1 d'un tuple.

'banane' étant la **deuxième** entrée du tuple, nous utiliserons le **deuxième** index pour atteindre cette valeur, soit l'index **1**.

L'exécution de notre code nous renverra donc la réponse suivante :

```
banane
```

Résultat de l'exécution du script précédent.

En Python, contrairement à d'autres langages, il est possible d'utiliser des **index négatifs**.

Ainsi, l'index -1 représentera le dernier élément d'une collection, -2 représentera l'avant-dernier et ainsi de suite.

Autre élément intéressant avec les tuples : il est possible de les **décomposer**. Décomposer un tuple consiste en la répartition du contenu du tuple dans N variables différentes. N représentant le nombre d'éléments présents dans le tuple.

Exemple de décomposition d'un tuple :

```
monTuple = ("pomme", "banane", "orange")
fruit1 , fruit2, fruit3 = monTuple

print("Le premier fruit est", fruit1)
print("Le deuxième fruit est", fruit2)
print("Le troisième fruit est", fruit3)
```

Script qui décompose un tuple de 3 éléments dans 3 variables distinctes.

Dans l'exemple, la règle **N éléments = N variables**, la décomposition du tuple se passe sans problème et l'exécution du code donne le résultat suivant :

```
Le premier fruit est pomme  
Le deuxième fruit est banane  
Le troisième fruit est orange
```

Résultat de l'exécution du script précédent.

Par contre, l'utilisation d'un nombre de variables inapproprié va générer une erreur :

```
monTuple = ("pomme", "banane", "orange")  
fruit1 , fruit2 = monTuple  
  
print("Le premier fruit est", fruit1)  
print("Le deuxième fruit est", fruit2)
```

Script qui répartit les éléments d'un tuple dans un nombre insuffisant de variables.

```
ValueError: too many values to unpack (expected 2)
```

Message d'erreur affiché lors de l'exécution du script précédent.

En français : Erreur de valeur : trop de valeurs à décomposer (2 attendues).



Opération sur les tuples

Il est possible de **concaténer deux tuples** au moyen de l'opérateur **+** afin d'en produire un nouveau tel qu'illustré dans l'exemple ci-dessous :

```
monTuple = ("pomme", "banane")
monTuple2 = ("orange",)

resultat = monTuple + monTuple2

print("Le resultat est :", resultat)
```

Script qui concatène deux tuples et affiche le résultat.

Notez la **présence d'un séparateur (virgule) au sein de monTuple2**.

Sans ce séparateur, monTuple2 serait considéré comme une simple chaîne de caractères et ne pourrait donc pas être concaténé ; la concaténation ne fonctionnant qu'entre tuples.

L'exécution de ce script donnera le résultat suivant :

```
Le résultat est : ('pomme', 'banane', 'orange')
```

Résultat de l'exécution du script précédent.

Il est également possible de **vérifier la présence d'un élément dans un tuple** au moyen de l'opérateur **in**.

Il est possible d'**itérer sur toutes les valeurs** du tuple au moyen d'une instruction **for** telle qu'illustré ci-après :

```
monTuple = ("pomme", "banane", "orange")

for value in monTuple:
    print("Valeur du tuple:", value)
```

Script qui affiche tous les éléments d'un tuple.

L'exécution de ce script donner le résultat suivant :

```
Valeur du tuple: pomme
Valeur du tuple: banane
Valeur du tuple: orange
```

Résultat de l'exécution du script précédent.

CONTINUER

Les dictionnaires

Définition

Un **dictionnaire** consiste en une collection de type « key-value pair » soit **couple clé-valeur**.

Il s'agit d'une structure de données permettant de stocker et d'organiser des éléments sous forme de couples. Une clé est créée et ajoutée au dictionnaire. Une valeur est associée à cette clé. La valeur est accessible au moyen de la clé.

```
"nom": "Dupont"
```

Exemple d'un couple clé-valeur.

L'**avantage du dictionnaire** est de **pouvoir stocker des valeurs de manière nominative et significative pour le contexte d'utilisation**. Il est possible, par exemple, de stocker au sein d'un dictionnaire un ensemble de propriétés telles que les caractéristiques d'une personne (clés nom, prénom, âge, etc), les propriétés d'un fichier (clés tailles, format, nombre de lignes, etc), etc.

Création d'un dictionnaire

Pour créer une collection de type dictionnaire, on utilise **les accolades { et }**.
Les paires clé - valeur sont séparées par le caractère double-point :

Exemple :

```
myDict = {"nom": "Doe", "prenom": "John"}  
print(myDict)
```

Script qui crée un dictionnaire et affiche son contenu.

Un dictionnaire contenant 2 clés (nom et prenom) ainsi que leurs valeurs respectives a été créé.

Le résultat de l'exécution du code sera le suivant :

```
{'nom': 'Doe', 'prenom': 'John'}
```

Résultat de l'exécution du script précédent.

Il existe une autre façon de créer un dictionnaire : en utilisant la fonction **dict()**.

Contrairement aux autres méthodes vues jusqu'ici, la fonction **dict()** est un peu particulière : au lieu de délimiter la clé par des guillemets ou apostrophes, nous allons simplement définir celle-ci **de la même manière que nous déclarons une variable** soit de la manière

suivante: **clé = "valeur"**. Les couples clé-valeur étant quant à eux toujours séparés par une virgule.

Exemple :

```
myDict = dict(nom = "Doe", prenom = "John")
print(myDict)
```

Script qui crée un dictionnaire au moyen de la fonction dict() et affiche le contenu de la collection.

Le résultat reste inchangé :

```
{'nom': 'Doe', 'prenom': 'John'}
```

Résultat de l'exécution du script précédent.

Manipulation d'un dictionnaire

Les dictionnaires, contrairement aux autres collections, **utilisent des chaînes de caractères pour l'indexation** : les clés associées aux valeurs !

Exemple :

```
myDict = dict(nom = "Doe", prenom = "John")
print("Le nom présent dans le dictionnaire est :", myDict["nom"])
```

Script qui crée un dictionnaire et affiche la valeur associée à la clé "nom".

Ce code nous fournira donc le résultat suivant :

```
Le nom présent dans le dictionnaire est : Doe
```

Résultat du script précédent.

Une deuxième méthode n'utilisant pas les *indexeurs* existe pour récupérer une valeur. Il s'agit de la méthode **get()**.

On l'utilise de la manière suivante :

```
monDictionnaire.get("clé")
```

Exemple :

```
myDict = dict(nom = "Doe", prenom = "John")  
print("Le nom présent dans le dictionnaire est :", myDict.get("nom"))
```

Script qui crée un dictionnaire et affiche la valeur associée à la clé "nom".



Notez que si vous tentez de récupérer la valeur d'une clé n'existant pas dans votre dictionnaire au moyen des indexeurs, une erreur sera levée lors de

l'exécution de votre code alors que si vous tentez la même chose au moyen de la méthode `get()` la valeur *None* sera retournée.

Opérations sur les dictionnaires

Il est possible de faire une multitude d'opérations sur un dictionnaire :

- Modifier la valeur associée à une clé.
- Ajouter un nouveau couple clé-valeur.
- Supprimer une clé du dictionnaire.
- Vérifier l'existence d'une clé.
- Parcourir les éléments du dictionnaire.

Modification de la valeur associée à une clé

Pour **modifier la valeur d'une clé d'un dictionnaire**, il est **impératif d'utiliser les indexeurs**.

Dans le cas de notre exemple, remplaçons la valeur associée à la clé "nom" du dictionnaire par la valeur "Smith" :

```
myDict = dict(nom = "Doe", prenom = "John")  
myDict["nom"]="Smith"  
print(myDict)
```

Script qui modifie la valeur associée à une clé.

Ce qui donne le résultat suivant :

```
{'nom': 'Smith', 'prenom': 'John'}
```

Résultat du script précédent.

Ajout d'un nouveau couple clé-valeur

Contrairement aux tuples qui sont immuables, **les dictionnaires peuvent être modifiés.**

Comme dans le cas d'une modification de valeur, les indexeurs sont à nouveau employés mais sans aucun risque d'erreur étant donné qu'on ne cherche pas à lire une clé mais bien à en ajouter une nouvelle.

Nous utiliserons dès lors le modèle suivant pour insérer un nouveau couple clé-valeur dans le dictionnaire :

```
monDictionnaire["clé à ajouter"] = "valeur associée"
```

Exemple :

```
myDict = dict(nom = "Doe", prenom = "John")
myDict["nom"]="Smith"
myDict["age"]=36
print(myDict)
```

Script qui ajoute une paire clé-valeur à un dictionnaire.

Lors de l'exécution du script, nous obtiendrons le résultat suivant :

```
{'nom': 'Smith', 'prenom': 'John', 'age': 36}
```

Résultat du script précédent.

Quel est le **risque** d'une telle opération sans forme de vérification préalable ? Tout simplement le fait de voir **la valeur** d'un couple clé-valeur, déjà présente dans le dictionnaire, **se faire écraser** par une autre lors de l'ajout d'une nouvelle clé.

Suppression d'une clé du dictionnaire

Tout comme la lecture d'une valeur du dictionnaire, la suppression d'une clé réagira de manière relativement désagréable si tant est que la clé n'est pas présente dans le dictionnaire.

Pour le reste, **si la clé est présente dans le dictionnaire**, l'utilisation de l'instruction **del** permet de supprimer définitivement l'entrée du dictionnaire.

L'utilisation de l'instruction **del** s'effectue de la manière suivante :

```
del monDictionnaire["clé à supprimer"]
```

Exemple :

```
myDict = dict(nom = "Doe", prenom = "John", age=36)

print("Dictionnaire AVANT suppression:", myDict)

del myDict["age"]

print("Dictionnaire APRES suppression:", myDict)
```

Script qui supprime la paire clé-valeur dont l'index est "age" et affiche la collection.

Après exécution du script, le dictionnaire ne contiendra plus la clé "age".

Résultat :

```
Dictionnaire AVANT suppression: {'nom': 'Doe', 'prenom': 'John', 'age': 36}
Dictionnaire APRES suppression: {'nom': 'Doe', 'prenom': 'John'}
```

Résultat du script précédent.

Vérification de l'existence d'une clé

Pour **vérifier l'existence d'une clé**, on utilise l'opérateur **in** suivi du nom de la clé.

Si l'expression renvoie la valeur **True** c'est que la clé est **présente**. Si l'expression renvoie **False** c'est qu'elle est **absente**.

Testons la présence d'une clé dans le dictionnaire 'myDict'. Nous allons chercher la clé 'nom' et ensuite nous assurer que la clé 'adresse' n'existe pas :

```
myDict = dict(nom = "Doe", prenom = "John", age=36)

if 'nom' in myDict:
    print("La clé nom est dans le dictionnaire")

if 'adresse' not in myDict:
    print("La clé adresse n'est pas dans le dictionnaire")
```

Script qui vérifie la présence et l'absence de clés dans un dictionnaire.

Résultat:

```
La clé nom est dans le dictionnaire
La clé adresse n'est pas dans le dictionnaire
```

Résultat du script précédent.

Lecture des éléments d'un dictionnaire

Il existe 3 manières de lire un dictionnaire au moyen d'une structure itérative for :

- Parcourir le dictionnaire **par clés** (méthode par défaut).
Lors des itérations, toutes les clés du dictionnaire sont lues, l'une après l'autre.
- Parcourir le dictionnaire **par valeurs**.
Lors des itérations, toutes les valeurs du dictionnaire sont lues, l'une après l'autre.
- Parcourir le dictionnaire **par couple clé-valeur**.
Lors des itérations, toutes les paires clé-valeur sont lues, l'une après l'autre.

1° Parcourir le dictionnaire par clés

Bien que ce soit la manière par défaut d'itérer sur un dictionnaire, il y a deux façons d'aborder la chose : avec ou sans l'utilisation de la méthode **keys()**.

```
myDict = dict(nom = "Doe", prenom = "John", age=36)

#sans keys()
for key in myDict:
    print("clé du dictionnaire :", key)

#avec keys()
for key in myDict.keys():
    print("clé du dictionnaire depuis keys() :", key)
```

Script qui parcourt un dictionnaire, clé par clé.

Le résultat sera le même dans les deux cas :

```
clé du dictionnaire : nom  
clé du dictionnaire : prenom  
clé du dictionnaire : age  
  
clé du dictionnaire depuis keys() : nom  
clé du dictionnaire depuis keys() : prenom  
clé du dictionnaire depuis keys() : age
```

Résultat du script précédent.

2° Parcourir le dictionnaire par valeurs

```
myDict = dict(nom = "Doe", prenom = "John", age=36)  
  
#avec values()  
for value in myDict.values():  
    print("valeur du dictionnaire :", value)
```

Script qui parcourt un dictionnaire, valeur par valeur.

Résultat :

```
valeur du dictionnaire : Doe  
valeur du dictionnaire : John  
valeur du dictionnaire : 36
```

Résultat du script précédent.

3° Parcourir le dictionnaire par couple clé-valeur

Dans ce cas, **deux valeurs seront récupérées lors de chaque itération**.

La première valeur est la **clé** du dictionnaire pour l'itération courante et la seconde valeur est la **valeur** associée à la clé.

Exemple :

```
myDict = dict(nom = "Doe", prenom = "John", age=36)

#avec values()
for key,value in myDict.items():
    print(f"valeur du dictionnaire pour la clé {key}: {value}")
```

Script qui récupère les paires clé-valeur d'un dictionnaire et les affiche.

Résultat :

```
valeur du dictionnaire pour la clé nom: Doe
valeur du dictionnaire pour la clé prenom: John
valeur du dictionnaire pour la clé age: 36
```

Résultat du script précédent.

CONTINUER

Review

Sélectionnez les affirmations correctes. Plusieurs réponses possibles.

☐

On peut définir un set en utilisant les accolades pour le délimiter: {"item"}

☐

On peut définir un set en utilisant les crochets pour le délimiter: ["item"]

☐

On peut définir un set au moyen de la fonction set()

☐

On peut fusionner un set et un dictionnaire au moyen de l'opérateur "|"

☐

La fonction tuple() permet de créer un tuple à partir d'une liste

SUBMIT

Associez chaque opérateur à sa méthode.

$\equiv$ |

union()

$\equiv$ &

intersection()

$\equiv$ -

difference()

SUBMIT

Soit les sets suivants :

monSet contenant les valeurs 1,2,5,6.

monSet2 contenant les valeurs 3,4,5,6.

monSet3 contenant les valeurs 5,6.

Écrivez le code permettant **d'unir** monSet et monSet2. Faites la **différence** entre monSet2 et monSet3. **Soustrayez** de monSet les valeurs de monSet3.



CONTINUER